Library Management

System

Final Project — Introduction to Programming 2

Case 6: Library Management System

Group Project | Python

Temirlan Dastan Dias Yerasyl

Problem Description

Book Tracking

Libraries must track which books are available, borrowed, or lost. Manual tracking leads to errors and confusion.

User Management

Different users have different borrowing limits and roles. Managing this manually is inefficient and error-prone.

Reporting & Stats

Without a system, generating borrow history, popular books, or overdue reports requires significant manual effort.

Project Structure

Modular folder structure

models/	book.py • user.py • transaction.py
services/	book_service.py • user_service.py • library_service.py
utils/	file_handler.py • validators.py
data/	books.json • users.json • history.json
tests/	test_library.py

main.py

Entry point — CLI menu

LibraryService

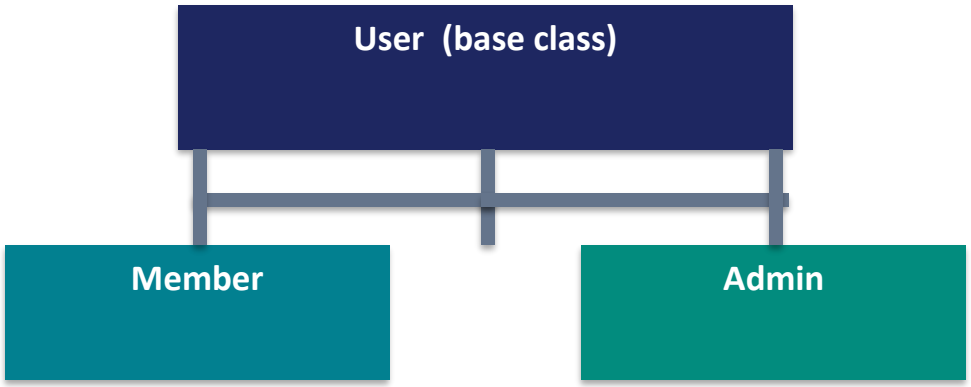
Orchestrates all logic

BookService

UserService

Object-Oriented Design

Class Hierarchy



Member attributes:

- membership_type: standard / premium
- borrow_limit: 3 or 7
- can_borrow() → checks limit

Admin attributes:

- borrow_limit: 10
- can_borrow() → override
- get_info() → role: admin

Other Classes

Book

book_id, title, author, available
to_dict() | available setter

BorrowRecord

user_id, book_id, borrow_date
mark_returned() | is_active()

✓ Encapsulation ✓ Inheritance ✓ Polymorphism

Key Features

Borrow & Return

Validates book availability, user borrow limits, and prevents borrowing the same book twice.

Search Books

Case-insensitive search by title or author using substring matching.

User Roles

Standard members (3 books), Premium (7 books), Admins (10 books).

JSON Storage

All data persisted to books.json, users.json, history.json automatically.

Statistics

Most borrowed book, active borrow count, total transaction history.

Unit Testing

unittest suite: Book, User, BorrowRecord, Validators — edge cases covered.

Algorithms & Data Structures

Data Structure Choices

dict[int, Book]	O(1) lookup by book ID — faster than list iteration O(n)
dict[int, User]	O(1) user access — no need to scan all users
list[BorrowRecord]	Ordered history — easy to iterate and filter
set (membership types)	O(1) membership validation with Python sets

Advanced Python Features

Generator	iter_books() uses yield Memory-efficient book iteration
Regex	is_valid_name() uses re.match() Clean input validation
Properties	@property for encapsulation Controlled attribute access
Type hints	All functions typed Better readability & IDE support

Team Contribution

Member 1	Member 2	Member 3	Member 4
Models Layer	**Services Layer**	**Utils & Data**	**Main & Tests**
• book.py — Book class & properties • user.py — User / Member / Admin • transaction.py — BorrowRecord	• book_service.py — CRUD for books • user_service.py — CRUD for users • library_service.py — borrow/return	• file_handler.py — JSON I/O • validators.py — input validation • data/*.json — initial datasets	• main.py — full CLI menu • test_library.py — unit tests • README.md — documentation

All members contributed equally — verified via GitHub commit history

Conclusions

All requirements met

Borrow/return, search, roles, file I/O, OOP, testing — all implemented.

Efficient design

Dictionary-based lookups ensure O(1) access for books and users.

Clean architecture

Separation into models / services / utils keeps code maintainable.

Robust validation

Validators and error handling prevent invalid data from entering the system.

Tested & verified

Unit tests cover core logic, edge cases, and invalid inputs.

Thank you! We are ready for questions.